Phoenix-RTOS

WSystem calls GetSetVariable

Wiktor Daszczuk wiktor.daszczuk@pw.edu.pl

Waldemar Grabski waldemar.grabski@pw.edu.pl

1 INTRODUCTION

In this exercise, you will add *a global* variable of type int *to the Phoenix-RTOS* system to store the value in the kernel context, and two system calls:

- *setVariable* – setting the value of *the variable* variable,
- *getVariable* – returning the value of variable *variable*.

To prepare the system image (kernel compilation and system image building) you need tools (toolchains directory) that must be on the path (PATH environment variable).

CONSIDERATIONS FOR TOOL CONSTRUCTION IN THE LAB

In the lab, the tools for compiling and building the system are located in the */home/public/daily/soi/toolchains directory*. Add this directory to the paths in the PATH environment variable by adding the following to the end of the *.profile* file in the user's home directory:

```
# set PATH so it includes /home/public/dzienne/soi/toolchains/i386-pc-
phoenix/i386-pc-phoenix/bin if it exists
if [ -d "/home/public/dzienne/soi/toolchains/i386-pc-phoenix/i386-pc-
phoenix/bin" ] ; then
    PATH="/home/public/dzienne/soi/toolchains/i386-pc-phoenix/i386-pc-
phoenix/bin:$PATH"
fi
```

2 ADDING SYSTEM CALLS

Information on how to implement system calls is provided in the *Phoenix-RTOS system documentation* phoenix-rtos-doc/README.md at master · phoenix-rtos/phoenix-rtos-doc (github.com). Please refer to:

- the way in which system calls are made,
- method of passing parameters and returning the result to/from the system call, please note that:
    - in kernel mode, all memory of the process calling the system call is available,
    - the parameters of the call are accessed through macros that operate on the stack of the process calling the system call.

To add a new system call:

- add a function declaration (which will be visible in the user's context) in the libphoenix library,
- add a system call to the list of available system calls,

- add a function definition (in kernel context) that executes a system call.

All file/directory paths in the following points are relative to the Phoenix-RTOS source code directory, by default it is *~/phoenix-rtos-project/*.

2.1 ADDING FUNCTION DECLARATIONS IN LIBPHOENIX LIBRARY

In *the libphoenix* library, in the *libphoenix/include/sys/* directory, create a new header file *getsetvariable.h* containing the declarations of the functions that will be called in the context of the user. The definition of this function will be created when compiling *the libphoenix* library on the basis of the names added to the list of system calls at 2.2 Modifying the system calls list.

Create a file*: libphoenix/inlude/sys/getsetvariable.h* and add function declarations for system calls in it.

```
extern void setVariable(int value);
extern int getVariable();
```

2.2 MODIFYING THE SYSTEM CALLS LIST

Add the names of the new system calls to the end of the list of system calls.

In the file *phoenix-rtos-kernel/include/syscalls.h* add system call names:

```
#define SYSCALLS(ID) \
    ID(debug) \
    //
    ID(sbi_getchar) \
    \
    ID(setVariable) \
    ID(getVariable)
```

2.3 ADDING A DEFINITION OF A FUNCTION THAT MAKES A SYSTEM CALL

Add:

- a global variable *variable* that holds a value,
- in the *_syscalls_init* function, initialize the global variable *variable* (assign the value 1000 to it),
- definitions of the functions that execute the system call, function names must be prefixed with *syscalls_*.

Note: global variables created in the kernel are not zero-initialized and are not zero-initialized, so it is necessary to initialize the global variable in the *_syscalls_init* function.

In the file: *phoenix-rtos-kernel/syscalls.c*, add the definition of the functions and the definition of the variable *variable*.

```
#define SYSCALLS_NAME(name)   syscalls_##name,
#define SYSCALLS_STRING(name) #name,

int variable;

void syscalls_setVariable(void* ustack)
{
    GETFROMSTACK(ustack, int, variable, 0);
}

int syscalls_getVariable(void* ustack)
{
    return variable;
}
```

In *the file: phoenix-rtos-kernel/syscalls.c* in the function *_syscalls_init* add initialization of the variable *variable*.

```
void _syscalls_init(void)
{
    lib_printf("syscalls: Initializing syscall table [%d]\n",
                sizeof(syscalls) / sizeof(syscalls[0]));
    variable = 1000;
}
```

3 ADDING A TESTING PROGRAM

Add two test programs: *setVariable*, *getVariable*. Each program will be in a separate subdirectory of the *_user* directory.

3.1 ADDING *SETVARIABLE* TESTING PROGRAM

Create a directory: *_user/setVariable*

In the *_user/setVariable directory*, create two files: *Makefile* and *main.c*.

File: *Makefile*

```
#
# Makefile for user application
#
# Copyright 2021 Phoenix Systems
#

NAME := setVariable
LOCAL_SRCS := main.c

include $(binary.mk)
```

File: *main.c*

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/getsetvariable.h>

int main(int argc, char** argv)
{
    if (argc != 2)
          return 1;
    int value = 0;
    value = atoi(argv[1]);
    setVariable(value);
    printf("Variable set to value: %d\n", value);
    return 0;
}
```

ADDING *GETVARIABLE* TESTING PROGRAM

Create a directory: *_user/setVariable*

In the *_user/getVariable directory*, create two files: *Makefile* and *main.c*.

File: *Makefile*

```
#
# Makefile for user application
#
# Copyright 2021 Phoenix Systems
#

NAME := getVariable
LOCAL_SRCS := main.c

include $(binary.mk)
```

File: *main.c*

```
#include <stdio.h>
#include <sys/getsetvariable.h>

int main(void)
{
    int value = 0;
    value = getVariable();
    printf("Read value: %d\n", value);
    return 0;
}
```